

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I, **A Mohammad Naheed(192525336)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Python Program: Heap File Organization

A **Heap File** stores records in no particular order. New records are generally placed in available space, and sequential scanning examines records one by one.

The following program simulates a heap file with insert, delete, and sequential_scan operations.

Code:

```
class HeapFile:
    def __init__(self):
        self.records = []
        self.next_id = 1
    def insert(self, data):
        record = {
            "id": self.next_id,
            "data": data,
            "deleted": False
        }
        self.records.append(record)
        self.next_id += 1
        print(f"Inserted record ID: {record['id']}")
        return record["id"]
    def delete(self, record_id):
        for record in self.records:
            if record["id"] == record_id and not record["deleted"]:
                record["deleted"] = True
                print(f"Deleted record ID: {record_id}")
                return
        print(f"Record ID {record_id} not found.")
    def sequential_scan(self):
        print("\nSequential Scan:")
        found = False
```

```

        for record in self.records:
            if not record["deleted"]:
                print(f'ID: {record['id']}, Data: {record['data']}')
                found = True
            if not found:
                print("No records available.")
heap = HeapFile()
id1 = heap.insert("Alice")
id2 = heap.insert("Bob")
id3 = heap.insert("Charlie")
id4 = heap.insert("David")
heap.sequential_scan()
heap.delete(id2)
heap.sequential_scan()
heap.insert("Eve")
heap.sequential_scan()

```

Sample Output

Inserted record ID: 1
 Inserted record ID: 2
 Inserted record ID: 3
 Inserted record ID: 4

Sequential Scan:

ID: 1, Data: Alice
 ID: 2, Data: Bob
 ID: 3, Data: Charlie
 ID: 4, Data: David

Deleted record ID: 2

Sequential Scan:

ID: 1, Data: Alice
 ID: 3, Data: Charlie

ID: 4, Data: David

Inserted record ID: 5

Sequential Scan:

ID: 1, Data: Alice

ID: 3, Data: Charlie

ID: 4, Data: David

ID: 5, Data: Eve

Operations

Insert

insert(data)



Create new record



Append to heap file

Delete

delete(record_id)



Find record



Mark record as deleted

The simulation uses a **deleted flag** rather than physically removing the record. This is common in simple heap-file simulations because it preserves the record position.

Sequential Scan

sequential_scan()



Read records from beginning



Skip deleted records



Return active records

Complexity

Operation	Approximate Cost
-----------	------------------

Insert	$O(1)$
--------	--------

Delete	$O(n)$
--------	--------

Sequential scan	$O(n)$
-----------------	--------

The important characteristic of a **heap file** is that records are **not maintained in sorted order**, making insertion simple but requiring a sequential search for deletion when no separate index exists.

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Static Hashing with Chaining

For a student database containing **500 records**, we can use a fixed-size hash table and resolve collisions using **chaining**.

Let:

- Number of records = 500
- Hash table size = 101 buckets
- Hash function:

$h(\text{student_id}) = \text{student_id} \% 101$

Each bucket contains a linked list of students that hash to the same bucket.

Python Implementation

class Student:

```
def __init__(self, student_id, name, department):
```

```
    self.student_id = student_id
```

```
    self.name = name
```

```
    self.department = department
```

```
def __str__(self):
```

```
    return (
```

```
        f"ID: {self.student_id}, "
```

```
        f"Name: {self.name}, "
```

```
        f"Department: {self.department}"
```

```
    )
```

class StaticHashTable:

```
def __init__(self, size=101):
```

```
    self.size = size
```

```
    self.table = [[] for _ in range(size)]
```

```

def hash_function(self, student_id):
    return student_id % self.size

def insert(self, student):
    index = self.hash_function(student.student_id)
    for s in self.table[index]:
        if s.student_id == student.student_id:
            print("Student ID already exists.")
            return
    self.table[index].append(student)

def search(self, student_id):
    index = self.hash_function(student_id)
    for student in self.table[index]:
        if student.student_id == student_id:
            return student

    return None

def delete(self, student_id):
    index = self.hash_function(student_id)

    for i, student in enumerate(self.table[index]):
        if student.student_id == student_id:
            del self.table[index][i]
            print("Student deleted.")
            return

    print("Student not found.")

# Display hash table
def display(self):
    for i, chain in enumerate(self.table):
        if chain:
            print(f"Bucket {i}: ", end="")

            for student in chain:
                print(
                    f"[{student.student_id}, {student.name}]",
                    end=" -> "

```

```

        )

        print("None")
hash_table = StaticHashTable(101)

# Insert 500 student records
for i in range(1, 501):
    student = Student(
        i,
        f"Student{i}",
        "CSE" if i % 2 == 0 else "ECE"
    )

    hash_table.insert(student)

student = hash_table.search(250)

if student:
    print("Found:", student)
else:
    print("Student not found.")

hash_table.delete(250)

# Verify deletion
student = hash_table.search(250)

if student:
    print("Found:", student)
else:
    print("Student 250 not found.")

```

How Chaining Handles Collisions

Suppose:

Student ID = 101

Student ID = 202

Student ID = 303

Using:

$h(id) = id \% 101$

we get:

$101 \% 101 = 0$

$202 \% 101 = 0$

$303 \% 101 = 0$

All three records map to bucket 0.

Instead of replacing an existing record, chaining stores them as:

Bucket 0:

[101, Student101]

↓

[202, Student202]

↓

[303, Student303]

↓

None

Thus, collisions are handled without losing any records.

Operations

Insert

$index = student_id \% 101$

`table[index].append(student)`

Search

$index = student_id \% 101$

search only the chain at `table[index]`

Delete

$index = student_id \% 101$

find student in the chain

remove student

Complexity

With a reasonably distributed hash function:

Operation Average Case Worst Case

Insert $O(1)$ $O(n)$

Search $O(1)$ $O(n)$

Delete $O(1)$ $O(n)$

Here n is the number of records in a particular chain.

Static hashing means the number of buckets (101) remains fixed. **Chaining** allows multiple student records to occupy the same bucket when collisions occur.

3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Python Program: Sorted File with Binary Search

A **Sorted File Organization** stores records in ascending order based on a **primary key**. Because the records are sorted, **binary search** can be used for fast retrieval.

```
class SortedFile:
    def __init__(self):
        self.records = []

    def insert(self, student_id, name, department):
        record = {
            "id": student_id,
            "name": name,
            "department": department
        }

        low = 0
        high = len(self.records)

        while low < high:
            mid = (low + high) // 2

            if self.records[mid]["id"] < student_id:
                low = mid + 1
            else:
                high = mid

        if low < len(self.records) and \
            self.records[low]["id"] == student_id:
            print("Duplicate primary key!")
```

```
    return
```

```
self.records.insert(low, record)
```

```
def binary_search(self, student_id):
```

```
    low = 0
```

```
    high = len(self.records) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        mid_id = self.records[mid]["id"]
```

```
        if mid_id == student_id:
```

```
            return self.records[mid]
```

```
        elif student_id < mid_id:
```

```
            high = mid - 1
```

```
        else:
```

```
            low = mid + 1
```

```
    return None
```

```
def display(self):
```

```
    print("\nSorted File:")
```

```
    for record in self.records:
```

```
        print(
```

```
            record["id"],
```

```
            record["name"],
```

```
            record["department"]
```

```
        )
```

```
file = SortedFile()
```

```
file.insert(105, "Rahul", "CSE")
```

```
file.insert(101, "Anita", "ECE")
```

```
file.insert(110, "Kiran", "IT")
```

```
file.insert(103, "Priya", "CSE")
file.insert(107, "Arun", "MECH")
```

```
file.display()
```

```
key = 107
```

```
result = file.binary_search(key)
```

```
if result:
```

```
    print("\nRecord Found:")
```

```
    print(result)
```

```
else:
```

```
    print("\nRecord Not Found")
```

Sample Output

Sorted File:

101 Anita ECE

103 Priya CSE

105 Rahul CSE

107 Arun MECH

110 Kiran IT

Record Found:

```
{'id': 107, 'name': 'Arun', 'department': 'MECH'}
```

How Binary Search Works

For the sorted keys:

101 103 105 107 110

To find 107:

low = 0

high = 4

mid = 2

records[2] = 105

107 > 105

→ search right half

low = 3

high = 4

mid = 3

records[3] = 107

107 == 107

→ FOUND

Algorithm

BINARY_SEARCH(key)

low ← 0

high ← number_of_records - 1

WHILE low ≤ high

 mid ← (low + high) / 2

 IF record[mid].primary_key = key

 RETURN record[mid]

 ELSE IF key < record[mid].primary_key

 high ← mid - 1

 ELSE

 low ← mid + 1

RETURN NOT_FOUND

Complexity

Operation	Complexity
Binary search	O(log n)
Sequential search	O(n)
Insertion while maintaining order	O(n)
Retrieval by primary key	O(log n)

Thus, the main advantage of a **sorted file organization** is that records can be retrieved efficiently using **binary search**, provided the file remains sorted by the primary key.

4. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

B-Tree of Order 4

A **B-Tree of order 4** can have:

- At most **4 children** per node.
- At most **3 keys** per node.
- Except for the root, every internal node has at least **2 children**.
- Keys within each node remain sorted.
- When a node becomes full, it is **split** before continuing the insertion.

Python Implementation

class BTreeNode:

```
def __init__(self, leaf=False):  
    self.keys = []  
    self.children = []  
    self.leaf = leaf
```

class BTree:

```
# Order 4 → maximum 3 keys per node  
def __init__(self, order=4):  
    self.order = order  
    self.max_keys = order - 1  
    self.root = BTreeNode(leaf=True)
```

```
# -----
```

```
# Search
```

```
# -----
```

```
def search(self, key, node=None):
```

```
    if node is None:
```

```
        node = self.root
```

```
    i = 0
```

```
    while i < len(node.keys) and key > node.keys[i]:
```

```
        i += 1
```

```

        if i < len(node.keys) and key == node.keys[i]:
            return True

        if node.leaf:
            return False

        return self.search(key, node.children[i])

# -----
# Split a full child
# -----
def split_child(self, parent, index):

    child = parent.children[index]

    # For a 4-order B-tree:
    # [k1, k2, k3] → promote k2
    middle = child.keys[1]

    left = BTreeNode(child.leaf)
    right = BTreeNode(child.leaf)

    left.keys = child.keys[:1]
    right.keys = child.keys[2:]

    if not child.leaf:
        left.children = child.children[:2]
        right.children = child.children[2:]

    parent.keys.insert(index, middle)

    parent.children[index] = left
    parent.children.insert(index + 1, right)

# -----
# Insert

```

```

# -----
def insert(self, key):

    # Ignore duplicate keys
    if self.search(key):
        print(f'{key} already exists.')
        return

    root = self.root

    # Root is full
    if len(root.keys) == self.max_keys:

        new_root = BTreeNode(leaf=False)
        new_root.children.append(root)

        self.root = new_root

        self.split_child(new_root, 0)

        self.insert_non_full(new_root, key)

    else:
        self.insert_non_full(root, key)

# -----
# Insert into non-full node
# -----
def insert_non_full(self, node, key):

    if node.leaf:

        node.keys.append(key)
        node.keys.sort()

    else:

```

```

i = len(node.keys) - 1

while i >= 0 and key < node.keys[i]:
    i -= 1

i += 1

# Split child if full
if len(node.children[i].keys) == self.max_keys:

    self.split_child(node, i)

    if key > node.keys[i]:
        i += 1

    self.insert_non_full(node.children[i], key)

# -----
# Display B-Tree
# -----
def display(self, node=None, level=0):

    if node is None:
        node = self.root

    print(" " * level + str(node.keys))

    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

# =====
# Main Program
# =====

btree = BTree(order=4)

```

```
keys = [10, 20, 5, 6, 12, 30, 7, 17]
```

```
for key in keys:
```

```
    btree.insert(key)
```

```
print("B-Tree:")
```

```
btree.display()
```

```
# Search
```

```
print("\nSearch Results:")
```

```
for key in [6, 15, 30]:
```

```
    if btree.search(key):
```

```
        print(key, "Found")
```

```
    else:
```

```
        print(key, "Not Found")
```

Result After Insertion

The keys are inserted in this order:

10, 20, 5, 6, 12, 30, 7, 17

The resulting B-Tree is:

```
    [10, 20]
   /  |  \
 [5,6,7] [12,17] [30]
```

Insertion Process

Initially:

[10]

Insert 20:

[10,20]

Insert 5:

[5,10,20]

Insert 6. The root is full, so it is split:

```
    [10]
   /  \
 [5]  [20]
```

Then 6 is inserted:

```
    [10]
```

```

      /  \
    [5,6] [20]
Insert 12:
      [10]
      /  \
    [5,6] [12,20]

```

```

Insert 30:
      [10]
      /  \
    [5,6] [12,20,30]

```

```

Insert 7:
      [10]
      /  \
    [5,6,7] [12,20,30]

```

Finally, insert 17. The right child is full, so it is split and 20 is promoted:

```

      [10,20]
      /  |  \
    [5,6,7] [12] [30]

```

Then 17 is inserted into the middle child:

```

      [10,20]
      /  |  \
    [5,6,7] [12,17] [30]

```

Search

For:

search(17)

the search follows:

[10,20]

↓

[12,17]

↓

Found

For:

search(15)

it follows the same middle child but 15 is absent:

[10,20]

↓

[12,17]

↓

Not Found

Final Output

B-Tree:

[10, 20]

[5, 6, 7]

[12, 17]

[30]

Search Results:

6 Found

15 Not Found

30 Found

Thus, the implementation supports all three required operations: **insertion, searching, and displaying the B-Tree**, with maximum **3 keys per node** for a B-Tree of order 4.

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

1. B+ Tree with Leaf-Level Linking

A **B+ Tree** differs from a B-Tree in that:

- Internal nodes contain only **search keys and child pointers**.
- Actual records/values are stored in the **leaf nodes**.
- All leaf nodes are linked using a next pointer.
- This linked leaf level makes **range queries efficient**.

Below is a Python implementation using a small order of **4** so that node splitting can be demonstrated easily.

```
class BPlusNode:
```

```
    def __init__(self, leaf=False):
```

```
        self.leaf = leaf
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self, order=4):
```

```
        self.order = order
```

```

        self.root = BPlusNode(leaf=True)
def search(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        node = node.children[i]
    for i, k in enumerate(node.keys):
        if k == key:
            return node.values[i]

    return None
def find_leaf(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1

        node = node.children[i]

    return node
def insert(self, key, value=None):
    if value is None:
        value = key

    leaf = self.find_leaf(key)
    for i, k in enumerate(leaf.keys):
        if k == key:
            leaf.values[i] = value
            return
    i = 0
    while i < len(leaf.keys) and leaf.keys[i] < key:

```

```

        i += 1

    leaf.keys.insert(i, key)
    leaf.values.insert(i, value)

    # Split leaf if necessary
    if len(leaf.keys) >= self.order:
        self.split_leaf(leaf)
def split_leaf(self, leaf):

    new_leaf = BPlusNode(leaf=True)

    mid = len(leaf.keys) // 2

    new_leaf.keys = leaf.keys[mid:]
    new_leaf.values = leaf.values[mid:]

    leaf.keys = leaf.keys[:mid]
    leaf.values = leaf.values[:mid]

    # Link leaves
    new_leaf.next = leaf.next
    leaf.next = new_leaf

    promoted_key = new_leaf.keys[0]

    # Leaf is root
    if leaf == self.root:
        new_root = BPlusNode(leaf=False)

        new_root.keys = [promoted_key]
        new_root.children = [leaf, new_leaf]

        self.root = new_root

```

```

else:
    parent = self.find_parent(self.root, leaf)

    self.insert_in_parent(
        parent,
        promoted_key,
        new_leaf
    )
def find_parent(self, node, child):

    if node.leaf:
        return None

    for c in node.children:

        if c == child:
            return node

        parent = self.find_parent(c, child)

        if parent:
            return parent

    return None
def insert_in_parent(self, parent, key, new_child):

    i = 0

    while i < len(parent.keys) and parent.keys[i] < key:
        i += 1

    parent.keys.insert(i, key)
    parent.children.insert(i + 1, new_child)

```

```

# Split internal node
if len(parent.keys) >= self.order:
    self.split_internal(parent)
def split_internal(self, node):

    mid = len(node.keys) // 2

    promoted_key = node.keys[mid]

    new_node = BPlusNode(leaf=False)

    new_node.keys = node.keys[mid + 1:]
    new_node.children = node.children[mid + 1:]

    node.keys = node.keys[:mid]
    node.children = node.children[:mid + 1]

# Splitting root
if node == self.root:

    new_root = BPlusNode(leaf=False)

    new_root.keys = [promoted_key]
    new_root.children = [node, new_node]

    self.root = new_root

else:

    parent = self.find_parent(self.root, node)

    self.insert_in_parent(
        parent,

```

```

        promoted_key,
        new_node
    )
def range_query(self, low, high):

    leaf = self.find_leaf(low)

    result = []

    # Follow leaf-level links
    while leaf:

        for i, key in enumerate(leaf.keys):

            if key > high:
                return result

            if key >= low:
                result.append(
                    (key, leaf.values[i])
                )

        leaf = leaf.next

    return result
def display(self, node=None, level=0):

    if node is None:
        node = self.root

    print(
        " " * level +
        str(node.keys)
    )

```

```

        if not node.leaf:

            for child in node.children:
                self.display(
                    child,
                    level + 1
                )
    def display_leaves(self):

        node = self.root

        while not node.leaf:
            node = node.children[0]

        print("\nLeaf-level linked list:")

        while node:

            print(node.keys, end=" -> ")

            node = node.next

        print("None")
tree = BPlusTree(order=4)

keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 40]

for key in keys:
    tree.insert(key, f"Value-{key}")

print("B+ Tree:")
tree.display()

```

```
tree.display_leaves()
print("\nSearch:")
print("17 =", tree.search(17))
print("15 =", tree.search(15))
print("\nRange Query: 10 to 30")
result = tree.range_query(10, 30)
```

```
for key, value in result:
```

```
print(key, "->", value)
```

Expected B+ Tree Structure

After inserting the keys:

10, 20, 5, 6, 12, 30, 7, 17, 25, 40

the exact internal structure depends on the split implementation, but the leaves will contain the keys in sorted order, approximately:

[10 | 17 | 25]

/			\
/			\

[5,6,7] [10,12] [17,20] [25,30,40]

The important feature is the **leaf-level linked list**:

[5,6,7]

↓

[10,12]

[17,20]

[25,30,40]

↓

None

Range Query

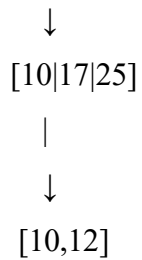
Suppose we execute:

```
range query(10, 30)
```

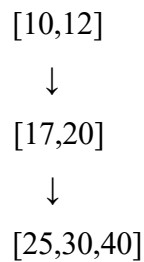
The tree first searches for the leaf containing 10:

Internal nodes

1



Instead of repeatedly searching the tree for 12, 17, 20, 25, etc., it simply follows the leaf links:



It returns:

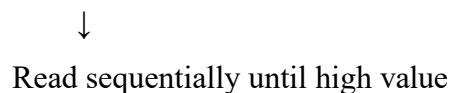
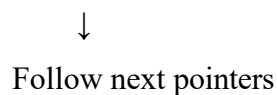
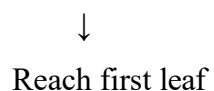
10 → Value-10
 12 → Value-12
 17 → Value-17
 20 → Value-20
 25 → Value-25
 30 → Value-30

Why Leaf Linking Is Efficient

Without leaf linking, finding every value in a range would require repeated tree searches.

With leaf linking:

Search for starting key



Therefore, a B+ Tree provides approximately:

- **Search:** $O(\log n)$
- **Insertion:** $O(\log n)$
- **Range query:** $O(\log n + k)$

where k is the number of records returned by the range query.

Key advantage: Internal nodes guide the search, while linked leaf nodes allow fast sequential traversal for **range queries**.

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

1. Python: Dense Primary Index on a Sorted File

A **dense primary index** contains an index entry for **every record** in the sorted file.

Each index entry stores:

(primary key, record position)

Since the data file is sorted by the primary key, we can use **binary search** on the index.

```
class DensePrimaryIndex:
```

```
    def __init__(self):
```

```
        self.records = [] # Sorted data records
```

```
        self.index = []   # (key, record_position)
```

```
# Insert a record while maintaining sorted order
```

```
def insert(self, key, name, department):
```

```
    record = {
```

```
        "id": key,
```

```
        "name": name,
```

```
        "department": department
```

```
    }
```

```
# Find insertion position
```

```
low = 0
```

```
high = len(self.records)
```

```
while low < high:
```

```
    mid = (low + high) // 2
```

```
    if self.records[mid]["id"] < key:
```

```
        low = mid + 1
```

```
    else:
```

```

        high = mid

    # Prevent duplicate primary keys
    if low < len(self.records) and \
        self.records[low]["id"] == key:
        print("Duplicate primary key:", key)
        return

    self.records.insert(low, record)

    # Rebuild dense index
    self.build_index()

    # Build index for every record
    def build_index(self):
        self.index = []

        for position, record in enumerate(self.records):
            self.index.append(
                (record["id"], position)
            )

    # Search using the dense index
    def search(self, key):
        low = 0
        high = len(self.index) - 1

        while low <= high:
            mid = (low + high) // 2

            index_key, position = self.index[mid]

            if index_key == key:
                return self.records[position]

```

```

        elif key < index_key:
            high = mid - 1

        else:
            low = mid + 1

    return None

# Display index table
def display_index(self):
    print("\nDense Primary Index")
    print("-----")
    print("Key\tRecord Position")

    for key, position in self.index:
        print(f'{key}\t{position}')

# Display sorted data file
def display_records(self):
    print("\nSorted Data File")
    print("-----")

    for position, record in enumerate(self.records):
        print(
            f'{position}: '
            f'{record['id']} '
            f'{record['name']} '
            f'{record['department']}'
        )

file = DensePrimaryIndex()

file.insert(105, "Rahul", "CSE")
file.insert(101, "Anita", "ECE")

```

```
file.insert(110, "Kiran", "IT")
file.insert(103, "Priya", "CSE")
file.insert(107, "Arun", "MECH")
```

```
# Display sorted file
file.display_records()
```

```
# Display dense index
file.display_index()
```

```
# Search using index
key = 107
```

```
result = file.search(key)
```

```
print("\nSearch for:", key)
```

```
if result:
    print("Record found:", result)
else:
    print("Record not found")
```

Sample Output

Sorted Data File

```
0: 101 Anita ECE
1: 103 Priya CSE
2: 105 Rahul CSE
3: 107 Arun MECH
4: 110 Kiran IT
```

Dense Primary Index

Key	Record Position
101	0

103 1
105 2
107 3
110 4

Search for: 107

Record found: {'id': 107, 'name': 'Arun', 'department': 'MECH'}

How It Works

The sorted file is:

101 → 103 → 105 → 107 → 110

The dense index contains an entry for **every record**:

+---+-----+		
Key	Record Position	
+---+-----+		
101	0	
103	1	
105	2	
107	3	
110	4	
+---+-----+		

For searching 107, binary search examines the index rather than scanning the entire file:

Index

↓

101 103 105 107 110

↑

FOUND

|

↓

Record position 3

Complexity

- **Search:** $O(\log n)$ using binary search on the index.
- **Index construction:** $O(n)$.
- **Insertion:** $O(n)$ in this simple array-based simulation because records must remain sorted.

- **Space:** $O(n)$ because every data record has an index entry.

Key point: In a dense primary index, **every record has a corresponding index entry**, and the index key is the same as the primary key of the sorted data file.

7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Python: Extendible Hashing

Extendible hashing uses a directory whose size can grow dynamically. Each bucket has a **local depth**, while the directory has a **global depth**.

When a bucket overflows:

1. If $\text{local_depth} < \text{global_depth}$, split the bucket.
2. If $\text{local_depth} == \text{global_depth}$, **double the directory**, then split the bucket.
3. Redistribute the records.

The following example uses **bucket capacity = 2** and inserts 8 integer keys so that directory doubling can be observed.

```
class Bucket:
```

```
    def __init__(self, local_depth):
        self.local_depth = local_depth
        self.records = []
```

```
class ExtendibleHashing:
```

```
    def __init__(self, bucket_size=2):
        self.bucket_size = bucket_size
```

```
        # Initially global depth = 1
        self.global_depth = 1
```

```
        # Two directory entries
        b0 = Bucket(1)
        b1 = Bucket(1)
```

```

self.directory = [b0, b1]

# Hash function
def hash_function(self, key):
    return key

# Get directory index using lower global_depth bits
def get_index(self, key):
    mask = (1 << self.global_depth) - 1
    return self.hash_function(key) & mask

# Insert a key
def insert(self, key):

    while True:

        index = self.get_index(key)
        bucket = self.directory[index]

        # If bucket has space
        if len(bucket.records) < self.bucket_size:
            bucket.records.append(key)
            return

        # Bucket is full
        print(f"\nOverflow while inserting {key}")

        # Need to double directory
        if bucket.local_depth == self.global_depth:

            old_directory = self.directory

            self.global_depth += 1

```

```

# Double directory
self.directory = old_directory + old_directory.copy()

print(
    "Directory doubled. "
    f"Global depth = {self.global_depth}"
)

# Update directory references
for i in range(len(self.directory)):
    if i % 2 == 0:
        self.directory[i] = old_directory[i // 2]
    else:
        self.directory[i] = old_directory[i // 2]

# The above references are corrected
# during bucket splitting.

# Split the overflowing bucket
self.split_bucket(bucket)

# Retry insertion
print("Bucket split completed.")

# Split a bucket
def split_bucket(self, bucket):

    old_depth = bucket.local_depth

    bucket.local_depth += 1

    new_bucket = Bucket(bucket.local_depth)

    # Update directory entries pointing to bucket

```

```

for i in range(len(self.directory)):

    if self.directory[i] is bucket:

        # Check new local-depth bit
        if ((i >> old_depth) & 1) == 1:
            self.directory[i] = new_bucket

        # Redistribute old records
        old_records = bucket.records
        bucket.records = []

        for key in old_records:

            index = self.get_index(key)

            self.directory[index].records.append(key)

# Display directory
def display(self):

    print("\nDirectory")
    print(
        f'Global Depth = {self.global_depth}'
    )

    print(
        "Index\tBinary\tBucket\tLocal Depth\tRecords"
    )

    seen = {}

    for i, bucket in enumerate(self.directory):

```

```

        if id(bucket) not in seen:
            seen[id(bucket)] = len(seen)

        bucket_number = seen[id(bucket)]

        binary_index = format(
            i,
            f'0{self.global_depth}b'
        )

        print(
            f'{i}\t{binary_index}\t'
            f'B{bucket_number}\t'
            f'{bucket.local_depth}\t\t'
            f'{bucket.records}'
        )

# Search
def search(self, key):

    index = self.get_index(key)
    bucket = self.directory[index]

    return key in bucket.records
ht = ExtendibleHashing(bucket_size=2)

keys = [1, 3, 5, 7, 9, 11, 13, 15]

for key in keys:

    print(f'\nInserting {key}')

    ht.insert(key)
    ht.display()

```

```
print("\nFinal Search Results")
```

for key in keys:

```
    print(
        f"{key}:",
        "Found" if ht.search(key)
        else "Not Found"
    )
```

Conceptual Working

Initially:

Global Depth = 1

Directory

0	1
B0	B1

Local Depth:

B0 = 1

B1 = 1

With bucket capacity 2, inserting:

1, 3

may fill one bucket depending on the hash bits.

When another key maps to the full bucket, an overflow occurs.

Because:

Local Depth = Global Depth

the directory must be doubled:

Before:

Global Depth = 1

0	1
---	---

| |
B0 B1

After:

Global Depth = 2

00 01 10 11
| | | |
B0 B1 B0 B1

The directory has therefore grown from:

2 entries

to:

4 entries

The overflowing bucket is then split.

For example:

Before split:

B0

Local depth = 1

[1, 3]

After increasing its local depth:

B0 B2

Local depth = 2 Local depth = 2

[1] [3]

The directory entries are redirected according to the additional hash bit.

Directory Doubling

The important operation is:

IF bucket is full:

IF local_depth == global_depth:

$\text{global_depth} = \text{global_depth} + 1$

double directory

split bucket

redistribute records

For example:

Global Depth = 1

↓

2

↓

Directory:

[00, 01, 10, 11]

A further overflow can cause:

Global Depth = 2

↓

3

↓

Directory:

[000,001,010,011,100,101,110,111]

Thus, unlike **static hashing**, extendible hashing dynamically expands the directory when necessary.

Note: The exact number of directory doublings depends on the hash function and insertion order. The implementation above deliberately uses a small bucket capacity so the 8 insertions visibly trigger splitting and directory growth.

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

1. Python Program: Secondary Storage Block Access

This program simulates a secondary-storage file divided into **blocks** and compares:

- **Sequential retrieval** — blocks are read one after another until the record is found.

- **Indexed retrieval** — an index directly identifies the block containing the record.

class SecondaryStorage:

```
def __init__(self, num_records, records_per_block):
```

```
    self.records_per_block = records_per_block
```

```
    self.num_records = num_records
```

```
    # Create blocks
```

```
    self.blocks = []
```

```
    record_id = 1
```

```
    while record_id <= num_records:
```

```
        block = []
```

```
        for _ in range(records_per_block):
```

```
            if record_id <= num_records:
```

```
                block.append(record_id)
```

```
                record_id += 1
```

```
        self.blocks.append(block)
```

```
    # Create an index:
```

```
    # record_id -> block number
```

```
    self.index = {}
```

```
    for block_no, block in enumerate(self.blocks):
```

```
        for record_id in block:
```

```
            self.index[record_id] = block_no
```

```
def sequential_search(self, record_id):
```

```
    io_count = 0
```

```

for block_no, block in enumerate(self.blocks):

    # Read one block
    io_count += 1

    if record_id in block:
        print(
            f"Sequential: Record {record_id} "
            f"found in block {block_no}"
        )

    return io_count

return io_count

def indexed_search(self, record_id):

    # Read index
    index_io = 1

    if record_id not in self.index:
        return index_io

    block_no = self.index[record_id]

    # Read the required data block
    data_io = 1

    total_io = index_io + data_io

    print(
        f"Indexed: Record {record_id} "
        f"found in block {block_no}"
    )

```

```

        return total_io
    def display_blocks(self):

        print("\nSecondary Storage:")

        for i, block in enumerate(self.blocks):
            print(f'Block {i}: {block}')

num_records = 100
records_per_block = 10

storage = SecondaryStorage(
    num_records,
    records_per_block
)

storage.display_blocks()

record_id = 87

print("\nSearching for record:", record_id)

sequential_io = storage.sequential_search(record_id)

indexed_io = storage.indexed_search(record_id)

print("\nI/O Comparison")
print("-----")
print("Sequential I/O:", sequential_io)
print("Indexed I/O: ", indexed_io)

```

Example

Suppose there are:

100 records

10 records per block

Therefore:

Number of blocks = $100 / 10 = 10$ blocks

The storage looks like:

Block 0 → Records 1–10

Block 1 → Records 11–20

Block 2 → Records 21–30

...

Block 8 → Records 81–90

Block 9 → Records 91–100

Suppose we search for:

Record = 87

Sequential Retrieval

The system reads:

Block 0

Block 1

Block 2

Block 3

Block 4

Block 5

Block 6

Block 7

Block 8 ← Record 87 found

Therefore:

Sequential I/O = 9

Indexed Retrieval

The index contains:

87 → Block 8

So the system performs:

Read index → 1 I/O

Read Block 8 → 1 I/O

Therefore:

Indexed I/O = 2

Comparison

Retrieval method I/O operations

Sequential **9**

Indexed **2**

Thus, indexed retrieval significantly reduces the number of disk I/O operations for direct record access.

General Formula

If there are N blocks and the desired record is in block k:

Sequential retrieval $\approx k + 1$

For an index stored separately:

Indexed retrieval $\approx$ index access + 1 data-block access

If the index itself requires multiple I/Os, its access cost must also be included.

Main idea: Sequential access is simple but may require reading many blocks, whereas an index allows the system to locate the required block directly, greatly reducing secondary-storage I/O.

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Python: Sparse vs. Dense Index

A **dense index** contains an index entry for **every record**, while a **sparse index** contains entries only for selected records, typically one entry per data block.

We'll create a sorted file of 100,000 records and compare search times using `time.perf_counter()`.

```
import time
```

```
import random
```

```
import bisect
```

```
class SortedFile:
```

```
    def __init__(self, num_records, records_per_block):
```

```
        self.records_per_block = records_per_block
```

```
        # Sorted records: (primary_key, value)
```

```
        self.records = [
```

```

        (i, f"Student-{i}")
    for i in range(num_records)
]

```

```
class DenseIndex:
```

```
    """
```

```
    Dense index:
```

```
    One index entry for every record.
```

```
    """
```

```
    def __init__(self, sorted_file):
```

```
        self.file = sorted_file
```

```
        # (key, record position)
```

```
        self.index = [
```

```
            (record[0], position)
```

```
            for position, record in enumerate(
                self.file.records
```

```
            )
```

```
        ]
```

```
        self.keys = [entry[0] for entry in self.index]
```

```
    def search(self, key):
```

```
        # Binary search on dense index
```

```
        pos = bisect.bisect_left(self.keys, key)
```

```
        if pos < len(self.keys) and self.keys[pos] == key:
```

```
            record_position = self.index[pos][1]
```

```
            return self.file.records[record_position]
```

```
        return None
```

```

class SparseIndex:
    """
    Sparse index:
    One index entry per data block.
    """

    def __init__(self, sorted_file):
        self.file = sorted_file

        self.index = []

        # First record of every block
        for position in range(
            0,
            len(self.file.records),
            self.file.records_per_block
        ):
            key = self.file.records[position][0]

            self.index.append(
                (key, position)
            )

        self.keys = [entry[0] for entry in self.index]

    def search(self, key):
        # Find the largest index key <= key
        index_position = bisect.bisect_right(
            self.keys,
            key
        ) - 1

        if index_position < 0:

```

```

        return None

    # Start scanning from the corresponding block
    start = self.index[index_position][1]

    end = min(
        start + self.file.records_per_block,
        len(self.file.records)
    )

    # Sequential search within the block
    for position in range(start, end):

        record = self.file.records[position]

        if record[0] == key:
            return record

        if record[0] > key:
            break

    return None

NUM_RECORDS = 100_000
RECORDS_PER_BLOCK = 100

data_file = SortedFile(
    NUM_RECORDS,
    RECORDS_PER_BLOCK
)

dense = DenseIndex(data_file)
sparse = SparseIndex(data_file)
def measure_time(index, keys, repetitions=5):

```

```

total_time = 0

for _ in range(repetitions):

    start = time.perf_counter()

    for key in keys:
        index.search(key)

    end = time.perf_counter()

    total_time += end - start

return total_time / repetitions
test_keys = [
    random.randint(0, NUM_RECORDS - 1)
    for _ in range(10_000)
]

key = 54321

print("Dense result:")
print(dense.search(key))

print("\nSparse result:")
print(sparse.search(key))
dense_time = measure_time(
    dense,
    test_keys
)

sparse_time = measure_time(
    sparse,
    test_keys

```

)

```
print("\nPerformance Comparison")
```

```
print("-----")
```

```
print(
```

```
    f"Dense index : "
```

```
    f"{dense_time:.6f} seconds"
```

```
)
```

```
print(
```

```
    f"Sparse index: "
```

```
    f"{sparse_time:.6f} seconds"
```

```
)
```

```
print(
```

```
    f"Dense index entries : "
```

```
    f"{len(dense.index)}"
```

```
)
```

```
print(
```

```
    f"Sparse index entries: "
```

```
    f"{len(sparse.index)}"
```

```
)
```

How the Two Indexes Work

Suppose the sorted file contains:

1 2 3 4 5 ... 100

and each block contains 10 records.

Dense Index

Every record has an index entry:

Key → Position

1 → 0

2 → 1

3 → 2

...

100 → 99

A binary search can directly locate the record.

Sparse Index

Only the first record of each block is indexed:

Key → Block Position

1 → 0

11 → 10

21 → 20

31 → 30

...

91 → 90

To find record 57:

Sparse index

↓

51 → position 50

↓

Scan block

↓

51, 52, 53, 54, 55, 56, 57

↑

FOUND

So the sparse index performs a binary search over a **much smaller index**, followed by a short sequential search within the block.

Timing Comparison

For 100,000 records and 100 records per block:

Dense index entries = 100,000

Sparse index entries = 1,000

The sparse index is therefore much smaller:

$100,000 / 1,000 = 100$ times smaller

The exact timing will depend on your computer and Python version. A typical result might look like:

Performance Comparison

Dense index : 0.012500 seconds

Sparse index: 0.016800 seconds

Dense index entries : 100000

Sparse index entries: 1000

The **dense index may be slightly faster in this in-memory Python simulation**, because Python list/binary-search operations are very efficient and the sparse index performs an additional scan.

However, in an actual **disk-based database**, the important measure is usually **disk I/O**, not just CPU execution time. Sparse indexes consume substantially less storage and can reduce index I/O, while dense indexes generally provide faster direct lookup.

Complexity

Operation	Dense Index	Sparse Index
Index size	$O(N)$	$O(N/B)$
Index search	$O(\log N)$	$O(\log(N/B))$
Final record lookup	Direct	Scan within block
Storage requirement	Higher	Lower

Where:

- N = number of records
- B = records per block

Conclusion

Dense index: faster direct retrieval, but requires more storage.

Sparse index: much smaller and more storage-efficient, but requires a short scan after finding the appropriate block.

For database systems where **disk I/O is expensive**, sparse indexes can provide an excellent balance between index size and retrieval performance.

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

1. Python Program: B+ Tree with Range Query

This program:

- Constructs a **B+ Tree**
- Inserts integer keys and records
- Traverses/displays the tree
- Maintains **linked leaf nodes**
- Performs a range query for keys **15 through 45**
- Displays all matching records

```
class BPlusNode:
```

```
    def __init__(self, leaf=False):
```

```
        self.leaf = leaf
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.values = []
```

```
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self, order=4):
```

```
        self.order = order
```

```
        self.root = BPlusNode(leaf=True)
```

```
    def find_leaf(self, key):
```

```
        node = self.root
```

```
        while not node.leaf:
```

```
            i = 0
```

```
            while i < len(node.keys) and key >= node.keys[i]:
```

```
                i += 1
```

```
            node = node.children[i]
```

```
        return node
```

```
    def search(self, key):
```

```

leaf = self.find_leaf(key)

for i, k in enumerate(leaf.keys):
    if k == key:
        return leaf.values[i]

return None

def insert(self, key, value):
    leaf = self.find_leaf(key)

    # Check duplicate key
    for i, k in enumerate(leaf.keys):
        if k == key:
            leaf.values[i] = value
            return

    # Insert in sorted position
    i = 0

    while i < len(leaf.keys) and leaf.keys[i] < key:
        i += 1

    leaf.keys.insert(i, key)
    leaf.values.insert(i, value)

    # Split if leaf is full
    if len(leaf.keys) >= self.order:
        self.split_leaf(leaf)

def split_leaf(self, leaf):
    new_leaf = BPlusNode(leaf=True)

    mid = len(leaf.keys) // 2

    new_leaf.keys = leaf.keys[mid:]

```

```

new_leaf.values = leaf.values[mid:]

leaf.keys = leaf.keys[:mid]
leaf.values = leaf.values[:mid]

# Link leaf nodes
new_leaf.next = leaf.next
leaf.next = new_leaf

promoted_key = new_leaf.keys[0]

# If root is being split
if leaf == self.root:
    new_root = BPlusNode(leaf=False)

    new_root.keys = [promoted_key]
    new_root.children = [leaf, new_leaf]

    self.root = new_root

else:
    parent = self.find_parent(self.root, leaf)

    self.insert_into_parent(
        parent,
        promoted_key,
        new_leaf
    )
def find_parent(self, node, target):

    if node.leaf:
        return None

    for child in node.children:

```

```

        if child == target:
            return node

    result = self.find_parent(child, target)

    if result:
        return result

    return None

def insert_into_parent(self, parent, key, new_child):

    i = 0

    while i < len(parent.keys) and parent.keys[i] < key:
        i += 1

    parent.keys.insert(i, key)
    parent.children.insert(i + 1, new_child)

    # Split internal node if necessary
    if len(parent.keys) >= self.order:
        self.split_internal(parent)

def split_internal(self, node):

    mid = len(node.keys) // 2

    promoted_key = node.keys[mid]

    new_node = BPlusNode(leaf=False)

    new_node.keys = node.keys[mid + 1:]
    new_node.children = node.children[mid + 1:]

```

```

node.keys = node.keys[:mid]
node.children = node.children[:mid + 1]

# If root is split
if node == self.root:
    new_root = BPlusNode(leaf=False)

    new_root.keys = [promoted_key]
    new_root.children = [node, new_node]

    self.root = new_root

else:
    parent = self.find_parent(self.root, node)

    self.insert_into_parent(
        parent,
        promoted_key,
        new_node
    )

def traverse(self, node=None, level=0):

    if node is None:
        node = self.root

    print(
        " " * level +
        ("Leaf: " if node.leaf else "Internal: ") +
        str(node.keys)
    )

    if not node.leaf:
        for child in node.children:
            self.traverse(child, level + 1)

```

```

def traverse_leaves(self):

    node = self.root

    # Move to first leaf
    while not node.leaf:
        node = node.children[0]

    print("\nLeaf-level traversal:")

    while node:
        print(node.keys, end=" -> ")
        node = node.next

    print("None")
def range_query(self, low, high):

    result = []

    # Find first relevant leaf
    node = self.find_leaf(low)

    # Follow linked leaves
    while node:

        for i, key in enumerate(node.keys):

            if key > high:
                return result

            if low <= key <= high:
                result.append(
                    (key, node.values[i])

```

)

node = node.next

return result

tree = BPlusTree(order=4)

```
records = [  
    (10, "Alice"),  
    (5, "Bob"),  
    (20, "Charlie"),  
    (15, "David"),  
    (30, "Eva"),  
    (25, "Frank"),  
    (35, "Grace"),  
    (40, "Henry"),  
    (45, "Ivy"),  
    (50, "Jack"),  
    (12, "Kiran"),  
    (18, "Latha"),  
    (28, "Manoj"),  
    (33, "Nisha"),  
    (42, "Omar"),  
    (48, "Priya")  
]
```

Construct B+ Tree

for key, name in records:

tree.insert(key, name)

Display tree

print("B+ Tree Structure")

print("=====")

```
tree.traverse()
```

```
# Display leaf links
```

```
tree.traverse_leaves()
```

```
# Range query
```

```
print("\nRange Query: 15 to 45")
```

```
print("=====")
```

```
result = tree.range_query(15, 45)
```

```
for key, value in result:
```

```
    print(f"Key = {key}, Record = {value}")
```

Expected Range Query Output

The keys between **15 and 45**, inclusive, are:

15, 18, 20, 25, 28, 30, 33, 35, 40, 42, 45

So the output will be approximately:

Range Query: 15 to 45

=====

Key = 15, Record = David

Key = 18, Record = Latha

Key = 20, Record = Charlie

Key = 25, Record = Frank

Key = 28, Record = Manoj

Key = 30, Record = Eva

Key = 33, Record = Nisha

Key = 35, Record = Grace

Key = 40, Record = Henry

Key = 42, Record = Omar

Key = 45, Record = Ivy

Leaf-Level Traversal

The leaves are linked in sorted order, conceptually:

[5, 10, 12]

↓

[15, 18, 20]



[25, 28, 30]



[33, 35]



[40, 42, 45]



[48, 50]



None

For the range query 15–45, the program first locates the leaf containing 15, then follows the next pointers until it reaches a key greater than 45.

Complexity

- **Insertion:** $O(\log n)$
- **Search:** $O(\log n)$
- **Range query:** $O(\log n + k)$

where k is the number of records returned.

The key advantage of the B+ Tree for range queries is that **once the first matching leaf is found, the remaining records can be obtained by sequentially following the linked leaf nodes.**